Reproducibility Guide:
Hybrid State-Dependent Ordering for Distributed Flatness

This guide is a step-by-step manual to reproduce the verification-first pipeline and experiment artifacts for:

"Hybrid State-Dependent Ordering for Distributed Flatness: Certified Switching and Jump Mitigation."

It is written so an independent reviewer can go from a clean checkout to regenerated figures/tables/artifacts.

1 Scope and Pipeline

The repository is organized around a verification-first pipeline:

1. **Gate R (Operating Envelope)**: Monte Carlo reachable trimmed set estimation (R).
2. **Gate 1 (Admissibility)**: DAG/topological consistency and switching well-posedness.
3. **Gate 2 (Flat Output Rank)**: decoupling/rank sanity check for the toy model.
4. **Gate 3 (Jump Diagnostics + Constants)**: J_{raw} vs. J, constants and provenance.
5. **Gate 3.5 (Dwell-Time Satisfiability)**: template inequality check from Gate 3 constants.
6. **Gate 4 (Sweeps)**: paired Monte Carlo sweeps over hysteresis (ϵ), lockout/dwell (τ_d), blending, and noise.

Optional supporting audits:

- **Gate S**: ordering metric sensitivity near ties.
- **Gate B**: blending transient quality inside blend windows.
- **Metric ablation**: robustness across ordering metrics.
- **Baseline demo**: fixed-order/no-switch baseline vs. hybrid switching.

2 Core Model and Logic (Code Map)

The core simulator is Python-based with second-order channels:

- Plant dynamics: `src/model/dynamics.py`
- Coupling/wake surrogate: `src/model/coupling.py`
- Coupling graph ($G(x)$): `src/model/graph.py`
- Forward-substitution/flatness recursion: `src/flatness/recursion.py`
- Hybrid ordering + hysteresis + lockout: `src/hybrid/ordering.py`
- Transition blending (jump mitigation): `src/hybrid/blending.py`
- Closed-loop simulator/log schema: `src/control/closed`$_{l}oop.py$

Interpretation:

- State is $x = [x1_1..x1_N,\ x2_1..x2_N]$.
- $x1$ behaves as the ordering coordinate (default metric $s_i = x1_i$).
- Dynamics are continuous-time with fixed-step discrete integration.
- Sample period is `dt` in config ($T_s = 0.02\,\mathrm{s}$ by default).
- Inversion computes u by forward-substitution over ordering π.
- On mode changes, applied control can be blended over transition horizon T_b.

3 Requirements and Environment

Minimum:

- macOS/Linux/WSL shell
- Python ≥ 3.9 (3.10+ recommended)
- `pip` + `venv`

  Dependencies are in `requirements.txt`:
- `numpy`, `scipy`, `pyyaml`, `matplotlib`, `networkx`

3.1 Create a clean virtual environment

```
cd hybrid_flatness_ext
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install --upgrade pip
python3 -m pip install -r requirements.txt
```

Optional deterministic plotting/cache environment (recommended in CI/headless):

```
export PYTHONPYCACHEPREFIX=/tmp/pycache
export MPLCONFIGDIR=/tmp/mpl
export XDG_CACHE_HOME=/tmp/xdg-cache
export MPLBACKEND=Agg
```

4 Configuration Files and Key Parameters

Primary configs:

- System and gate defaults: `configs/system.yaml`
- Gate 4 sweep definition: `configs/experiments.yaml`

Important defaults in `configs/system.yaml`:

- `system.N: 3`
- `system.dt: 0.02` (sample period T_s)
- Wake surrogate:
  - `system.wake_decay_L` (L)
  - `system.wake_Rx` (R_x)
  - `system.gamma_edge` (γ_{edge})
  - `system.k_wake`
- Hybrid:
  - `ordering.epsilon` (ϵ, hysteresis margin)
  - `ordering.lockout_sec` (τ_d, minimum time between switches)
  - `ordering.blend_sec`/`ordering.transition_blend_sec` (T_b)

5 Quickest End-to-End Reproduction

Run the full artifact export pipeline:

```
bash scripts/make_success_eval.sh
```

This runs gates and writes the paper-style bundle to:

- `paper_artifacts/success_eval/`

Equivalent direct command:

```
python3 -m tools.export_success_eval --out paper_artifacts/success_eval
```

6 Gate-by-Gate Reproduction (CLI)

All commands assume you are inside `hybrid_flatness_ext` with venv active.

6.1 Gate R: reachable trimmed set (operating envelope)

Manuscript-style convenience command:

```
python3 run_gate.py --gate R --samples 25 --method bbox
```

If your environment maps `python` to the same interpreter:

```
python run_gate.py --gate R --samples 25 --method bbox
```

Canonical module command:

```
python3 -m src.verify.reachset --config configs/system.yaml --samples 25 --method bbox
```

Outputs:

- `results/reachset/R_bounds.json`
- `results/reachset/reachset.json`
- `results/reachset/reachset_points.npz`

Optional boundary invariance check over R:

```
python3 run_gate.py --gate R_env
# or
python3 -m src.verify.gateR_envelope_invariance --config configs/system.yaml
```

Outputs under `results/gateR/`.

6.2 Gate 1: admissibility (DAG + topological consistency)

```
python3 run_gate.py --gate 1 --config configs/system.yaml
# or
python3 -m src.verify.gate1_graph --config configs/system.yaml
```

Negative-vignette export (if a failing episode is found):

```
python3 run_gate.py --gate 1 --export_negative_vignette --vignette_graph_mode
    physical_all_edges
```

Outputs:

- `results/gate1/gate1_summary.json`
- `results/gate1/gate1_rates.png`
- `results/gate1/gate1_interswitch_hist.png`
- optional `results/vignette_negative/*`

6.3 Gate 2: flat output rank regularity

```
python3 run_gate.py --gate 2
# or
python3 -m src.verify.gate2_flat_output_rank --config configs/system.yaml
```

Output:

- `results/gate2/gate2_rank_summary.json`

6.4 Gate 3: jump diagnostics and constants

```
python3 run_gate.py --gate 3
# or
python3 -m src.verify.gate3_constants --config configs/system.yaml
```

Outputs:

- `results/gate3/constants_table.json`
- `results/gate3/mismatch_vs_rho_loglog.png`
- `results/gate3/jump_histogram.png`
- `results/gate3/u_udot_timeseries.png`

6.5 Gate 3.5: dwell-time satisfiability inequality

```
python3 run_gate.py --gate 3.5
# or
python3 -m src.verify.gate3p5_satisfiability --config configs/system.yaml
```

Output:

- `results/gate3p5/gate3p5.json`
- `results/gate3p5/satisfiability_vs_tau_d.png`

Checked condition (template form in code):

- `lhs = mu * exp(-(alpha_over_c2) * Kd) < 1`
- with `tau_d = Kd * dt` from Gate 3 constants provenance.

6.6 Gate 4: Monte Carlo sweeps (ϵ, τ_d, blending, noise)

Manuscript-style convenience command:

```
python3 run_experiments.py --config configs/experiments.yaml
```

Equivalent form (if `python` resolves correctly):

```
python run_experiments.py --config configs/experiments.yaml
```

Canonical module command:

```
python3 -m src.verify.gate4_stability_inequality --config configs/experiments.yaml
```

Output:

- `results/gate4/gate4_summary.csv`
- `results/gate4/*.png` (design/sweep plots)
- `results/gate4/epsilon_tradeoff.json`

Optional self-consistency test:

```
python3 run_experiments.py --config configs/experiments.yaml --run_gate4_selftest
# or
python3 -m src.verify.gate4_selftest --config configs/experiments.yaml
```

6.7 Optional Gate S and Gate B audits

Gate S (ordering sensitivity near ties):

```
python3 run_gate.py --gate S
# or
python3 -m src.verify.gateS_sensitivity --config configs/system.yaml
```

Gate B (blend transient quality):

```
python3 run_gate.py --gate B
# or
python3 -m src.verify.gateB_blend_transient --config configs/system.yaml
```

7 Plot Reproduction

Most plots are generated automatically by each gate command.

To regenerate all key paper-style plots in one pass:

```
python3 -m tools.export_success_eval --out paper_artifacts/success_eval
```

Key plot locations:

- Gate 1: `results/gate1/*.png`
- Gate 3: `results/gate3/*.png`
- Gate 3.5: `results/gate3p5/*.png`
- Gate 4: `results/gate4/*.png`
- Gate R: `results/gateR/*.png`
- Gate S: `results/gateS/*.png`
- Gate B: `results/gateB/*.png`
- Metric ablation: `results/metric`$_{a}blation/*.png$
- `Baseline demo: results/baseline`$_{d}emo/*.png$

8 Artifact Validation Against `paper_artifacts/success_eval`

The committed success bundle includes:

- run manifest + git/config hashes
- snapshot configs
- source-file hash snapshot
- generated gate outputs/plots

Reference files:

- `paper_artifacts/success_eval/run_manifest.json`

- `paper_artifacts/success_eval/snapshot/configs/system.yaml`
- `paper_artifacts/success_eval/snapshot/src_versions.txt`

Generate a fresh bundle and compare:

```
python3 -m tools.export_success_eval --out paper_artifacts/success_eval_repro
diff -rq paper_artifacts/success_eval paper_artifacts/success_eval_repro
```

Notes:

- `run_manifest.json` can differ in `git_head`/`git_dirty`.
- Small numeric variation can occur if config/code/seed changes.
- For strict replication, verify config hashes and source hashes match snapshot.

9 Metric Ablation Audit (`s_metric` robustness)

Run with wrapper:

```
python3 run_experiments.py --run_metric_ablation --metric_config configs/system.yaml --
    metric_out results/metric_ablation
```

Or module:

```
python3 -m src.experiments.metric_ablation --config configs/system.yaml --out results/
    metric_ablation
```

Outputs:

- `results/metric`$_a blation/metric_t able.csv$
- `results/metric`$_a blation/metric_a blation.json$
- `comparison plots in the same folder`

10 Python API Usage (Programmatic Runs)

Example: run Gate 4 and read the CSV.

```
from pathlib import Path
from src.verify.gate4_stability_inequality import run_gate4

csv_path = run_gate4("configs/experiments.yaml")
print("Gate4 CSV:", Path(csv_path).resolve())
```

Example: run all verification-first gates in Python:

```
from src.verify.reachset import run_reachset
from src.verify.gate1_graph import run_gate1
from src.verify.gate2_flat_output_rank import run_gate2
from src.verify.gate3_constants import run_gate3
from src.verify.gate3p5_satisfiability import run_gate35
from src.verify.gate4_stability_inequality import run_gate4

run_reachset("configs/system.yaml", samples=25, method="bbox")
```

```
run_gate1("configs/system.yaml")
run_gate2("configs/system.yaml")
run_gate3("configs/system.yaml")
run_gate35("configs/system.yaml")
run_gate4("configs/experiments.yaml")
```

11 Troubleshooting

- If `python` is not found: use `python3` explicitly.
- If plots fail in headless mode: set `MPLBACKEND=Agg` and rerun.
- If Gate 3.5 errors on missing constants: run Gate 3 first so `results/gate3/constants_table.json` exists.
- If Gate R invariance errors on missing bounds: run Gate R reachset first (`src.verify.reachset`).
- If metric ablation reports assertion errors for specific metrics: this is expected for some non-robust metric choices and is recorded in `metric_ablation` outputs.

12 Suggested Reviewer Workflow

1. Create venv and install dependencies.
2. Run `python3 -m tools.export_success_eval -out paper_artifacts/success_eval_repro`.
3. Compare `paper_artifacts/success_eval_repro` against committed `paper_artifacts/success_eval`.
4. Re-run individual gates with modified ϵ, τ_d, and `transition_blend_sec` to test design sensitivity.
5. Run metric ablation to evaluate ordering-metric robustness beyond the default `s_metric`.